FestPass: event registration and a graded CI project

A Java starter for third-year computer science students. Create multiple events, book and cancel seats, and save registrations between runs. Jenkins runs 28 tests, builds a runnable JAR, and smoke-tests the real executable before publishing it. The starter deliberately fails tests; students diagnose and repair it.

One-hour Jenkins assessment: students create an SCM job, add the missing polling trigger, and diagnose the seeded defect for automatic red/green builds. No new Java feature or tests are required. See Student_Assignment.pdf or festpass-java/ASSIGNMENT.md for acceptance criteria and the 100-point rubric. Allow 60 minutes after Java/Git, Jenkins/plugins, the agent, and Git server are ready. No green starter build is required. The alias comes from the checked-out commit's Git author; fill only the issued class/challenge codes in submission.properties. Failed runs archive diagnostic evidence, not a JAR. Job creation, trigger configuration, and diagnosis are assessed work. The existing booking and storage code is supplied, not assigned work.

Two ZIPs: extract FestPass_Java_Project.zip and Jenkins_Laptop_Setup.zip into the same parent folder. Neither ZIP includes third-party executables, credentials, an existing Git history, or a configured Jenkins home.

Jenkins laptop lab: commit -> test -> build -> JAR

This folder contains **setup scripts and instructions**, not the Jenkins binary, plugins, a JDK, passwords, or an already configured Jenkins home. Download the software from the official links below. First-time setup needs internet access; after installation, the example can run entirely on one laptop.

The matching project is `festpass-java`. Use only your own trusted teaching code. This is a local learning environment, not a production Jenkins setup. The upgraded starter manages events, bookings, cancellations, and persistent data. The one-hour task in `festpass-java/ASSIGNMENT.md` is Jenkins-focused: create the SCM job, configure polling, and diagnose the seeded test failure before demonstrating automatic red/green builds. No new Java feature or tests are required. Complete sections 1-6 before the assessed hour; sections 7-9 are the student's work.

1. Prerequisites and official downloads

Item	Requirement	Official download / reference
Java	Full **JDK 21**, 64-bit; not just a JRE. JDK 25 is also supported by this pinned Jenkins release.	https://adoptium.net/temurin/releases/?version=21
Jenkins	LTS WAR file. This kit pins **2.580.1**, the LTS release checked on **October 25, 2026**.	https://www.jenkins.io/download/
Exact Jenkins WAR	Our download script downloads this official file and checks its SHA-256.	https://get.jenkins.io/war-stable/2.580.1/jenkins.war
Jenkins Java policy	Check this before substituting other Java or Jenkins releases. Java 17 is not sufficient for the pinned release.	https://www.jenkins.io/doc/book/platform-information/support-policy-java/
Git	Already installed; ensure command-line Git includes `git daemon`.	https://git-scm.com/downloads
IDE	Already installed; use any Java-capable IDE or a text editor. A C++ toolchain is not used.	No new IDE required.
Terminal	Windows PowerShell 5.1+; or Bash + curl on macOS/Linux. macOS uses `shasum`; Linux usually has `sha256sum`.	Usually supplied by the OS / Git installation.
Browser	Any recent desktop browser.	No browser extension required.
Jenkins plugins	Git (`git`), Pipeline (`workflow-aggregator`), JUnit (`junit`) and their dependencies.	https://plugins.jenkins.io/

No Maven, Gradle, Docker, Node.js, Python, database, GitHub account, public repository, cloud subscription, or credit card is needed.

Suggested lab allowance (not a vendor minimum): an 8 GB RAM laptop, around 2 GB free disk, and 30-60+ minutes for first-time downloads/configuration. The guided red/green exercise takes about 15-20 minutes once setup is ready. These scripts cap the controller heap at 768 MB and agent heap at 512 MB; the IDE, compiler, OS, and browser need additional memory.

The required local ports are **8080** (Jenkins HTTP) and **9418** (read-only Git daemon). Both servers bind to **127.0.0.1 only**. Do not expose them through port forwarding, tunnels, public Wi-Fi interfaces, or firewall rules. The inbound agent uses WebSocket over 8080; port 50000 is not required.

2. Extract the two ZIPs together

Extract the project ZIP and setup ZIP into the **same parent directory**. Avoid nesting one folder inside the other.

If you completed the standalone prerequisites, extract into your existing `CloudJenkinsLab` parent folder under your user home. Keep the existing `jenkins-lab/downloads` and `jenkins-lab/runtime` directories: the setup ZIP contains neither, so your Jenkins account/plugins are preserved. Stop your preparation Jenkins terminal before starting it again with the launcher. Skip re-downloading or repeating the wizard if already completed.

Windows example:

```
C:\CloudJenkinsLab\
  festpass-java\
    Jenkinsfile
    ci\Build.java
    src\...
  jenkins-lab\
    Download-Jenkins.ps1
    Start-Jenkins.ps1
    Start-GitServer.ps1
    Start-Agent.ps1
    ...
```

macOS/Linux: use a directory such as `$HOME/CloudJenkinsLab` with the same two child folder names. ZIP extraction may remove executable permissions; the instructions explicitly call `bash`, so `chmod` is not required.

3. Check Java and run the app first

Install Temurin **JDK 21** for your OS and CPU (x64 versus ARM64). On Windows, enable the installer options for PATH and JAVA_HOME, then open a new terminal. If you installed a ZIP build, set JAVA_HOME to the JDK directory and put its `bin` directory on your user PATH. Do not point JAVA_HOME at `bin`.

In every terminal that will start Jenkins or the agent, verify:

```
java -version
javac -version
jar --version
git --version
```

`java` and `javac` should both report 21.x (or both 25.x if deliberately using that alternative). An IDE can use a different bundled JDK; the **terminal versions** determine this lab's behavior.

Open a terminal in `festpass-java`:

```
java ci/Build.java verify
```

The starter deliberately fails tests. This is expected, not a JDK/setup issue. Do not bypass tests or repair the source before the assessed Jenkins diagnosis. The following app walkthrough is for AFTER the repair passes all 28 tests:

```
java ci/SmokeTest.java
java -jar build/festpass.jar add-event java "Java Workshop" 1
java -jar build/festpass.jar book java S1
java -jar build/festpass.jar book java S2
java -jar build/festpass.jar cancel java S1
java -jar build/festpass.jar book java S2
java -jar build/festpass.jar summary
```

After repair, expected: **28 tests, 0 failures**, a JAR, and seven passing real-JAR smoke checks. The first S2 booking deliberately fails with SOLD OUT (exit 2); continue with cancellation, then S2 succeeds. Final summary: one event, one booked seat, zero available. The app persists to `data/festpass.tsv`. For a repeat walkthrough, choose a new `--data FILE` before each command or use the existing events instead of trying to create duplicate IDs. If Git needs an identity, set your own name/email in this repository. Keep generated output out of Git; `.gitignore` is supplied. Before the first Jenkins run, fill `submission.properties` with your instructor-issued class code and unique challenge code, then commit it. The class code must be the same across the cohort. Read the assignment's build evidence notice; do not use passwords as codes. The pipeline reads your alias from the checked-out Git commit's author name, not the properties file or the Jenkins agent's Git configuration. Check `git config user.name` and `git log -1 --format=%an` in your original project. If you change your local Git name, make a new commit before Jenkins runs. Create the initial Git commit now, even though tests are failing:

```
git init -b main
git add .
git commit -m "Add diagnostic FestPass starter"
```

4. Terminal A: serve the committed Git repository

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-GitServer.ps1
```

macOS/Linux, from `jenkins-lab`:

```
bash start-git-server.sh
```

The execution-policy flag applies only to that PowerShell process. If school policy blocks local scripts, ask the instructor/administrator rather than changing machine-wide security settings.

Keep this terminal open. In another terminal, check:

```
git ls-remote git://127.0.0.1:9418/festpass-java
```

You should see a commit hash and `refs/heads/main`. This service publishes only this explicitly exported repository and permits read/clone/fetch, not push. It does not send code to an external service. No credentials are needed.

Do not use a local filesystem URL in the Jenkins job or disable Jenkins' local-checkout security protection. The loopback Git protocol avoids that workaround. If the folder name or port changes, use the URL printed by the launcher everywhere below.

5. Terminal B: download and start Jenkins

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Download-Jenkins.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-Jenkins.ps1
```

macOS/Linux:

```
bash download-jenkins.sh
bash start-jenkins.sh
```

Keep the terminal open. Visit **http://127.0.0.1:8080/**. The setup password is printed during startup and is stored at:

```
jenkins-lab/runtime/jenkins-home/secrets/initialAdminPassword
```

Unlock Jenkins, choose **Install suggested plugins**, and create your own administrator account. Keep authentication enabled. Set the Jenkins URL to `http://127.0.0.1:8080/`. Do not share your password or upload Jenkins home.

Under **Manage Jenkins -> Plugins**, verify these are installed and enabled:

- **Git** (`git`)

- **Pipeline** (`workflow-aggregator`)

- **JUnit** (`junit`)

Install missing entries and their dependencies, then restart if requested. `plugins.txt` documents the required plugin IDs; it is not a preinstalled plugin bundle. The exact UI labels can vary slightly across Jenkins releases.

6. Add a separate local build agent

Do not run these builds on the controller's built-in node.

1. Open **Manage Jenkins -> Nodes -> Built-In Node -> Configure**. Set **Number of executors = 0**, and save.

2. Select **New Node**, name it exactly **student-laptop**, and choose **Permanent Agent**.

3. Set **Number of executors = 1**.

4. Set **Remote root directory** to an absolute path dedicated to the lab:

- Windows: `C:\CloudJenkinsLab\jenkins-lab\runtime\agent-work`

- macOS/Linux: your actual `/Users/.../CloudJenkinsLab/jenkins-lab/runtime/agent-work` or `/home/.../CloudJenkinsLab/jenkins-lab/runtime/agent-work`. Do not type `$HOME` literally into the Jenkins UI.

5. Set **Labels = student-laptop**.

6. Set usage to **Only build jobs with label expressions matching this node**.

7. Choose **Launch agent by connecting it to the controller**. Use WebSocket if the UI offers it. Save.

8. Open the node page. Copy **only the agent secret** from its launch command. This secret is a credential: never commit it, screenshot it, or share it.

Terminal C: connect the agent

Windows PowerShell, from `jenkins-lab`:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\Start-Agent.ps1
```

macOS/Linux:

```
bash start-agent.sh
```

Paste the node secret at the hidden prompt. The launcher downloads `agent.jar` from your local Jenkins, passes the secret via a private temporary file, and removes that file when the process exits normally. If the process is forcibly killed, remove leftover `runtime/agent-private/secret-*` files after confirming the agent has stopped.

Keep Terminal C open. The Jenkins node page should change to **online**. The agent inherits this terminal's PATH: it must be able to run Git and the full JDK.

Security boundary: this lab uses separate processes/workspaces on one trusted laptop. If both run as your same OS account, this is **not strong OS-level isolation**: do not build untrusted repositories or pull requests. Production installations need stronger identity/isolation controls. See: https://www.jenkins.io/doc/book/security/controller-isolation/

7. Create the Pipeline-from-SCM job

1. On the Jenkins dashboard, select **New Item**.
2. Name: **festpass-ci**. Type: **Pipeline**.
3. In the Pipeline section set **Definition = Pipeline script from SCM**.
4. **SCM = Git**.
5. **Repository URL =** `git://127.0.0.1:9418/festpass-java`.
6. **Credentials = none**.
7. **Branch Specifier =** `*/main`.
8. **Script Path =** `Jenkinsfile` (case-sensitive on some systems).
9. Save the job. Before the first run, configure the trigger below.

The starter intentionally has no trigger. In the original repository's `Jenkinsfile`, add this block inside `pipeline`, beside `options` and `stages`:

```
triggers {
    pollSCM('* * * * *')
}
```

This checks Git once per minute and builds on detected changes, not on every timer tick. Do not substitute `cron` or enable Build periodically. Do not configure a second copy of this trigger in the UI. Commit the Jenkinsfile (`git add Jenkinsfile`, then `git commit -m "Configure Jenkins SCM polling"`), and select **Build Now once**.

The first run loads the Jenkinsfile and establishes its polling trigger. Expect stages **Checkout -> Submission setup -> Test -> Build JAR -> Smoke test JAR -> Save artifact**. The console prints the checked-out commit ID and JDK versions. Open the completed build's **Test Result** to inspect the 28 starter tests and their failures. The first run should FAIL at Test; product stages are skipped. It still archives `submission-evidence.csv`. A JAR becomes available only after a successful repair. A Submission setup failure means the issued codes are missing or invalid; correct the setup before diagnosing application tests.

If an optional stage-visualization plugin is absent, the console, test result, build status, and artifacts still demonstrate the pipeline. No extra UI plugin is required for this kit.

8. Make a commit and watch Jenkins react

Make a harmless text change in the `help` output in `src/main/java/FestPass.java`. Keep command names and tested output fragments unchanged. Save and commit from the **original project folder**, not the Jenkins workspace:

```
git add src/main/java/FestPass.java
git commit -m "Personalize the help introduction"
```

After you add and bootstrap its trigger, the Jenkinsfile polls SCM once each minute. Usually the new build starts within one to two minutes; scheduling or a busy agent can delay it.

No push is needed in this laptop-only arrangement: Jenkins fetches your repository's committed history through the local read-only Git server. File edits without a commit do not trigger it. With a hosted Git remote, commits would also need to be pushed.

Compare `git log -1 --oneline` with the commit printed in Jenkins. This harmless change should trigger an automatic FAILED run because the seeded bug is still present. Wait for it to finish before committing a fix.

9. Red -> green exercise

1. Inspect the automatic red run in Test Result / Console Output.

2. Read a failed assertion and the related source. Identify the violated rule; the starter already contains the defect, so do not add another one.

3. Make the smallest application fix. Leave all tests and pipeline checks intact.

4. Run `java ci/Build.java verify` and `java ci/SmokeTest.java` locally.

5. Commit the fix and wait for an automatic green build with 28 passing tests.

6. Download the successful JAR and both red/green `submission-evidence.csv` files.

Build JAR, Smoke test JAR, and Save artifact must be skipped on test failure. The always-run evidence archive is separate: a failed run publishes evidence, not a runnable JAR. Do not interpret that evidence file as a successful build.

This demonstrates **CI and build artifact creation**, not deployment. The app is intentionally not a real multi-user reservation service.

10. Troubleshooting

Symptom	Check / fix
`java` / `javac` not found	Install the full JDK, fix PATH, open fresh terminals. An IDE's bundled Java may not be on PATH.
Unsupported Java / class-file version	Use terminal Java 21 for this pinned Jenkins. Java 17 is too old. Check both controller and agent terminals.
Port 8080 already in use	Choose e.g. 8081 in `Start-Jenkins.ps1 -Port 8081` or `bash start-jenkins.sh 8081`; use the same port in the browser, Jenkins URL, and agent launcher.
Git daemon cannot bind 9418	Stop only your own old lab server, or choose a free port and update the job URL.
`git daemon` is unavailable	Ensure your Git distribution includes it; Linux distributions may package it separately. Ask the instructor/admin if installation is restricted.
Repository not exported / connection refused	Keep Terminal A open, create the initial commit, check extraction paths, and run `git ls-remote` against the printed URL.
Cannot use `git://` under institutional policy	Do not bypass policy. Use an approved remote with the instructor, or Build Now with that approved SCM; hosted remotes need a push.
Waiting for an executor	Agent must be online with label `student-laptop`; terminal C stays open. Keep built-in executors at zero.
Agent unauthorized	Copy the node's **agent secret**, not your administrator password; verify the node name and port.
Git or Java missing only in Jenkins	Restart the agent from a terminal with the correct PATH.
Commit does not trigger a build	Add/commit the missing pollSCM trigger and bootstrap once manually; verify repo/branch, live Git server, and Polling Log. Allow scheduling time.
Jenkins tests old code	Compare commit hashes. Do not edit its workspace. Check the repo URL and `main` branch.
No JUnit step / unknown `pipeline` syntax	Install/enable the listed plugins and dependencies; restart as needed.
No test report after compilation error	Fix the compiler error first; compilation failed before the test runner could write XML.
Downloads fail behind a proxy	Use your institution's approved proxy/certificate setup. Do not disable TLS verification.

11. Stop and clean up

Stop **agent terminal C**, then **Jenkins terminal B**, then **Git terminal A** using Ctrl+C. Do not kill unrelated Java processes. No Windows services, scheduled tasks, Docker containers, or public cloud resources were created.

For another session, restart Git, Jenkins, and the agent; the Jenkins home preserves jobs and accounts. The agent secret can be found on its node page. When completely finished, remove only your extracted lab folder after saving anything you want to keep and ensuring its processes have stopped. Never distribute `runtime`, `downloads`, `.git`, passwords, or your configured Jenkins home with a class ZIP.

Official documentation

- Jenkins WAR installation: https://www.jenkins.io/doc/book/installing/war-file/

- Jenkinsfile: https://www.jenkins.io/doc/book/pipeline/jenkinsfile/

- Pipeline syntax and polling: https://www.jenkins.io/doc/book/pipeline/syntax/

- Agents: https://www.jenkins.io/doc/book/using/using-agents/

- JUnit plugin: https://plugins.jenkins.io/junit/
- Git daemon: https://git-scm.com/docs/git-daemon
- Java source-file launch: https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html